PROJECT COMPANION | F:\Steganography

From Zero to nsF5

Steganography

Information Hiding x Machine Learning x Engineering

A beginner-friendly handbook for the nsF5 image steganography project: start with pixels and bits, learn LSB hiding, chi-square and RS steganalysis, Hamming matrix embedding, F5/nsF5 and wet paper coding, SHA-256 keying, and supervised machine-learning detection - then finish with C++/GPU engineering and your own capstone experiments.

Suggested pace: 12 weeks x 6-8 hours per week

For undergraduates new to Python, security, and machine learning

Method: read the intuition -> open the project -> run experiments

September 6, 2026 | Project v1.4.0 (dual-version ML models)

Introduction - How to Use This Handbook

This is a handbook that teaches through code. It does not replace a textbook or the project thesis. Instead, it arranges the information-hiding and machine-learning ideas behind F:\Steganography in an order a beginner can actually follow: intuition and examples first, then the project implementation, then hands-on experiments.

0.1 Who This Is For

Assumed starting point - the handbook fills every other gap for you:

- Comfort with a Windows PC and installing software;
- Some college math exposure, or willingness to skip a formula and return later;
- A little programming is helpful but not required - Chapter 2 brings Python up to working level;
- Twelve weeks of 6-8 hours per week.

> **Tip |** If you have never installed Python, do not worry: the first two weeks are designed exactly for that situation. The goal is not to become a programmer but to become someone who can read and modify this project.

0.2 Reading Conventions

Marker	Meaning
Tip	Plain-language explanation or analogy that removes a common stumbling block
Watch out	A trap beginners frequently hit: types, mismatched parameters, misread statistics
Try it	A hands-on experiment you must run; reading alone does not count as

Marker	Meaning
	learning
Think about it	An open question with no single right answer, used to test real understanding
Read the code	Open a specific file and map the concept onto the implementation

0.3 The 12-Week Roadmap

The route follows a natural order: carrier basics -> steganography -> steganalysis -> machine learning -> engineering -> capstone. Every week maps to runnable code or a reproducible experiment:

Week	Topic	Chapter	Hands-on outcome
1	Digital images and binary	Ch. 1	Inspect pixels and bit planes yourself
2	Python / NumPy / Pillow	Ch. 2	Run the GUI or run_e2e.py; read/write image arrays
3-4	LSB hiding and blind steganalysis	Ch. 3	Hide a message, then detect it with chi-square/RS
5	Matrix embedding and F5	Ch. 4	Use Hamming codes to change less and hide more
6	nsF5, wet paper, hash keying	Ch. 5-6	Explain the wet-paper solver in ns5_core.py
7	Machine-learning foundations	Ch. 7	Explain features, labels, overfitting, AUC
8-9	ML-based steganalysis	Ch. 8	Run the v1 and v2 pipelines; explain SRM and dual models
10	C++ / GPU / data engineering	Ch. 9	Understand acceleration, self-checks, multi-source data
11-12	Capstone and presentation	Ch. 10-11	Complete and present a small

Week	Topic	Chapter	Hands-on outcome
			improvement experiment

Try it | Copy this table into a weekly checklist (Appendix C has a ready-made one). Tick a row every weekend and answer that chapter's "Think about it" question.

0.4 What You Will Be Able to Do

- Explain steganography, steganalysis, embedding efficiency, shrinkage, wet paper coding, and syndromes in your own words;
- Say why naive LSB hiding is exposed by chi-square and RS analysis;
- Work one p=3 Hamming embedding example by hand (at most one coefficient changed per block);
- Explain the real difference between F5 and nsF5, and why eliminating shrinkage matters;
- Explain how image-hash keying gives decoder self-synchronization, and where its limits are;
- Explain the full supervised-learning pipeline, including why photo-grouped cross-validation is essential;
- Understand the v1 11-D features and the v2 143-D features, including SRM statistics and the 143d/53d dual-model strategy;
- Explain what C++ DLLs and GPU code accelerate and why bit-level consistency checks matter;
- Complete a small, honest improvement experiment and write up the results.

0.5 Project Map: Know the Code Before You Study It

The project is a research tool: it runs algorithm experiments and ships as a GUI application. Remember these files first; every chapter returns to them:

File	Role	Chapter
src/image_io.py	Image I/O wrappers for 8-bit grayscale and color images	Ch. 2
src/ns5_core.py	Algorithm core: Hamming codes, wet paper, hash keying, embed/extract	Ch. 3-6
src/steganalysis.py	Blind steganalysis: chi-square, RS, combined verdict	Ch. 3
src/efficiency.py	Theoretical and measured code-family / efficiency curves	Ch. 4
src/matrix_demo.py	Teaching demo of syndrome lookup	Ch. 4
src/fsfeatures.py + cpp/fsfeatures.dll	ctypes binding for the v1 11-D features	Ch. 8
src/featurize_v2.py + src/srm_filter.py	v1.4: SRM preprocessing and the 143-D v2 feature set	Ch. 8
src/make_dataset.py / train_model.py	Dataset generation (v1/v2, SRM, multi-source) and training	Ch. 8
src/ml_predict.py	Single-image ML prediction (143d/53d dual models + sensitivity)	Ch. 8
src/cppembed.py + cpp/nsf5embed.dll	C++ embed/shuffle hot path and self-checks	Ch. 9
gpu/*.py	PyTorch batched features (v1 11-D / v2 143-D) and GPU training	Ch. 9
src/gui.py	Tkinter GUI with teaching panels	Ch. 9
src/test_core.py and friends	Regression tests for embedding, analysis, false positives	All

File	Role	Chapter

> **Read the code |** Keep the project window open from now on. Whenever the handbook names a file, switch to it. From Chapter 2 onward most topics assume you run before you read.

0.6 Learning Tips

- **Run first, understand later.** End-to-end scripts remove most paper confusion;
- **Digest formulas twice.** First pass: conclusion and intuition. Second pass: derivation;
- **Retell in your own words.** If you can explain a chapter summary to a friend, you own it;
- **Keep an experiment log.** Parameters and outputs are the most valuable material for your capstone;
- **Trust the tests.** test_core.py and test_steg.py are the fastest judge of whether you broke something.

> **Think about it |** Before you start, spend five minutes answering: what is the difference between encryption and steganography? If you cannot say, that is perfect - Chapter 3 starts there.

0.7 What Changed for v1.4.0 (2026-09-06)

The first draft of this handbook tracked project v1.3. On September 6 the project moved to v1.4.0, and this edition is synchronized with it. Chapters 1-6 (the steganography algorithms) are unchanged; the machine-learning and engineering chapters now include sections 8.8 and 9.7, and older scores are labeled as "v1 baselines".

- ML detection grew from 11-D features with LR/XGB (AUC ~0.75-0.79) to a 143-D LightGBM default (0.9085 average) plus a 53-D interpretable model (0.9227); see 8.8;
- New SRM high-pass preprocessing, 143-D v2 features, 12 embedding variants, real JPEG clean samples, and a full BOSSbase multi-source dataset; see 8.8 and 9.7;
- The repository moved to a new GitHub home (Yukinoshita-lin/nsf5-steganography) and the license is now Apache-2.0 with a NOTICE file;
- The glossary, command sheet, weekly checklist, and concept-to-code map were extended for v1.4.

Chapter 1 - Digital Images and Binary (Week 1)

> **Try it |** Goals: cement the intuition that an image is a matrix of numbers; do binary and byte arithmetic by hand; explain why both the eye and statistics tolerate small pixel perturbations.

1.1 A Digital Image Is Just a Grid of Numbers

Zoom into an **8-bit grayscale image** and you will find a two-dimensional table of integers from 0 to 255: 0 is black, 255 is white, and the values between them are shades of gray. The computer stores the dimensions and the numbers - not the picture your eye sees.

A **color image** is three such tables stacked together: R (red), G (green), and B (blue). Each pixel holds three 0-255 values; an orange pixel might be R=255, G=128, B=0.

In code, a grayscale image is a 2-D array and a color image is a 3-D array. The project wraps this in `src/image_io.py`: `load_as_gray()` reads any supported image as a grayscale array, and `save_image()` writes an array back to disk.

> **Tip |** Mathematics says rows and columns; image processing says height and width. A 512x512 image has 512 rows and 512 columns, so the array shape is (512, 512) - rows first.

1.2 Binary, Bytes, and Bits

A computer only understands 0 and 1. One 0/1 value is **1 bit**; eight bits form **1 byte**, which can represent 2^8 = 256 combinations - exactly the grayscale range 0-255.

For example, decimal 200 = 128 + 64 + 8, which is binary `11001000`. Its **least significant bit (LSB)** is the rightmost 0. Flipping 200 to 201 changes only the LSB, and the human eye cannot tell grayscale 200 from 201.

The LSB is the single most important concept in this handbook: LSB steganography is simply hiding secret bits inside the LSBs of pixels.

> **Try it |** Open a Python prompt (or a scratch script) and verify this for yourself:

Experience: decimal, binary, and the LSB

```
v = 200
print(bin(v))            # 0b11001000
print(v & 1)             # lowest bit = 0
print(v ^ 1)             # flip lowest bit = 201
print(v & 0b11111110)    # clear lowest bit = 200
print(v & 0b11111110 | 1)# clear then set = 201
```

1.3 Why Images Have Room to Hide Things

Images are full of **redundancy**, in two senses:

- **Visual redundancy:** the eye is insensitive to tiny brightness changes and fine detail; changing one pixel from 128 to 129 is nearly invisible;
- **Statistical redundancy:** neighboring pixels in natural images are highly correlated; formats like JPEG remove this redundancy to shrink files;
- **The LSB plane looks like noise:** the lowest bits of a natural photo already look random, which gives a natural camouflage environment.

Compression and steganography therefore fight over the same resource: compression wants to remove redundancy; steganography wants to use it as hiding space. This is also why many hiding algorithms are designed for a specific file format.

> **Watch out |** Do not confuse "invisible" with "safe." Invisibility is only the weakest requirement. A real analyst uses statistics, and naive LSB replacement leaves very clear statistical traces - exactly what chi-square and RS analysis catch in Chapter 3.

1.4 Bit Planes: Split an Image into 8 Layers

Write each grayscale value as 8 bits, then group pixels by bit position. You get eight binary images called **bit planes**. The most significant plane (MSB) carries the coarse structure; the LSB plane looks like scattered noise.

The intuition follows immediately: since the LSB plane is already near-random, inserting pseudo-random secret bits is hard to spot by eye or by simple histograms. But that same randomization changes internal structure, and smarter statistical tools can measure it.

Counting pixels and LSB values (numpy operations used everywhere in the project)

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)   # (2, 2) uint8
print(img & 1)                # LSB plane [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0]  <- MSB first
```

1.5 Back to the Project: Meet cover and stego

Steganography uses two fixed names: the original carrier is the **cover**; the image after hiding is the **stego**. The project uses them everywhere: `img/cover.png` is the demo carrier and `output/stego_*.png` files are embedded results.

> **Read the code |** Open step 1 of `src/run_e2e.py`: numpy builds a smooth 256x256 demo image with `np.linspace` gradients plus small noise and saves it as `img/cover.png`. Understanding that snippet means understanding how a natural-looking image grows out of numbers.

1.6 Summary and Self-Check

- A grayscale image is a 0-255 integer matrix; a color image has three channel matrices;

- The LSB is the lowest bit of a pixel value, and flipping it is visually negligible;
- Compression removes redundancy while steganography exploits it - they are opposing forces;
- cover = original carrier; stego = image after embedding.

> **Think about it |** Is a pure black image (all zeros) a good cover for LSB hiding? What about a pure-noise image? Write your guesses down and return after Chapter 3.

Chapter 2 - Tooling: Python, NumPy, and Image I/O (Week 2)

> **Try it |** Goals: install the environment, run the end-to-end script, and read/write image arrays with NumPy. From now on every experiment can be verified immediately in the project.

2.1 Setting Up

Dependencies are light: NumPy and Pillow for the core, plus scikit-learn/joblib/matplotlib for ML and plotting. Use Python 3.9+ (the README examples use 3.14; older versions work too).

Install in the project root and run the first self-tests

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # core algorithm self-test
python src\test_steg.py      # steganalysis self-test
python src\run_e2e.py        # embed -> decode -> analyze -> plot
```

> **Watch out |** If the default `python` lacks tkinter, the GUI will not start; use an interpreter that ships tkinter (README suggests `C:\Python314\python.exe`). Command-line scripts work without it. On `ModuleNotFoundError`, first confirm you installed into the interpreter that runs the script: `python -m pip install ...` is the safe form.

2.2 Thinking in NumPy

Four NumPy concepts cover almost everything: **shape**, **dtype**, **slicing**, and **vectorized operations**. Steganography algorithms really just: take an array, rewrite the LSBs of some positions in some order, and store the array back.

Ten lines covering the four concepts

```
import numpy as np
a = np.arange(12).reshape(3, 4)     # 3 rows, 4 columns
print(a.shape, a.dtype)             # (3,4) int64
b = a[:, 1]                          # second column
c = a[0:2, 0:3]                      # top-left 2x3 block
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255                 # draw a white square
print((x & 1).sum())                 # count LSB=1 pixels
```

Note `dtype=np.uint8`: pixels are unsigned 8-bit integers 0-255. Arithmetic wraps around: `np.uint8(200) + 100` gives 44, not 300. The project flips LSBs with XOR 1 rather than +/- 1, precisely to avoid wrapping 0 below zero or 255 above the range.

2.3 Image I/O: Meet image_io.py

The core interface of src/image_io.py

```
from PIL import Image
import numpy as np


def load_as_gray(path):
    im = Image.open(path).convert("L")   # force grayscale
    return np.asarray(im, dtype=np.uint8)


def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)
```

> **Read the code |** Read the whole file yourself and compare `load_image()` with `load_as_gray()`, plus the supported extensions. Then run this snippet on the project's own demo image:

Try it: read an image, inspect its shape, save a copy

```
import sys; sys.path.insert(0, "src")
import image_io as IO
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype)          # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")
```

2.4 First End-to-End Run: run_e2e.py

Run `python src\run_e2e.py`. The script: generates a cover image -> embeds an English message with nsF5 p=3 (empty and non-empty password) -> decodes and compares -> runs blind analysis on clean and stego images -> plots the code-family/efficiency curves.

> **Try it |** Read the console output carefully and copy three numbers: embedded bits, changed pixels, and the analysis probability for the clean versus stego image. By the end of Chapter 3 you will explain why they differ.

You do not need to understand embedding internals yet. This week's goal is only that the code runs on your machine and that the "change an LSB" intuition feels real.

2.5 Common Errors at a Glance

Error / symptom	Cause	Fix
ModuleNotFoundError	Installed into the wrong interpreter	python -m pip install ... with the interpreter you run
image too small to hold header + payload	Pixels below message capacity	Use a larger image, smaller p, or shorter message
UnicodeDecodeError / garbage	Default embedding supports ASCII only	Keep test messages ASCII; see Chapter 10 for UTF-8 extension ideas
GUI exits instantly / TclError	Interpreter has no tkinter	Run src/gui.py with a tkinter-enabled Python

> **Think about it |** Why does `image_io` call `np.ascontiguousarray` before saving? Hint: slices and transposes can produce non-contiguous views, and many low-level libraries dislike them.

Chapter 3 - LSB Steganography and Its Statistical Fingerprint (Weeks 3-4)

> **Try it |** Goals: complete the smallest possible "hide then detect" loop; understand what chi-square and RS analysis each measure; read the verdict logic in steganalysis.py.

3.1 Encryption vs. Steganography: The First Question

Dimension	Encryption	Steganography
Protects	Message content: unreadable without the key	The act of communication: nobody notices a secret exists
Visibility	Ciphertext clearly looks abnormal	The carrier looks completely normal
Typical use	Passwords, certificates, secure files	Covert channels, watermarking, forensic research
Relationship	Encrypt first, then hide (recommended)	Hiding conceals the existence of the secret

> **Tip |** A useful mnemonic: **encryption protects the content; steganography protects the existence.** Professional systems use both: encrypt the message first, then hide the ciphertext. Ciphertext looks random, which actually makes it blend in better statistically.

3.2 A Minimal Runnable LSB Embedder

The most naive method is **LSB replacement**: write the secret bit stream into pixel LSBs one bit at a time. A grayscale image with M x N pixels holds up to M x N bits.

Textbook minimal LSB (for understanding only; the project uses matrix coding from Chapters 4-5)

```
import numpy as np
```

```
def text_to_bits(s):
    raw = s.encode("ascii")
    # 16-bit length header + 8 bits per character
    head = [(len(raw) >> i) & 1 for i in range(16)]
    body = np.unpackbits(np.frombuffer(raw, np.uint8))
    return np.concatenate([head, body]).astype(np.uint8)


def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k]   # clear LSB, write bit
    return flat.reshape(cover.shape)


def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
    return bytes(np.packbits(flat[16:16 + nbytes*8]))
```

> **Watch out | Lossy formats are the enemy of LSB hiding.** PNG and BMP store pixels losslessly, so the bit planes survive; JPEG recompression destroys LSBs. The project therefore saves PNG by default, and JPEG-domain hiding needs a separate design (the yccstego extension works on quantized DCT coefficients).

Why start every message with a 16-bit length header? The decoder must know how many bytes to read. `encode_string()` in `src/ns5_core.py` does exactly this: 16 little-endian length bits, then the payload bits. You just re-implemented that structure in the snippet above.

3.3 Why It Gets Caught: The Chi-Square Test

In natural images, adjacent grayscale levels 2i and 2i+1 (say 100 and 101) usually occur with **unequal** frequencies. LSB replacement forces the pair counts toward equality: whenever a pixel moves from 2i to 2i+1, or vice versa, the two histogram counts flatten. Westfeld's chi-square test measures exactly this:

1. Build the 256-bin grayscale histogram and pair levels (0,1), (2,3), ..., (254,255);

2. Under the "embedded" hypothesis each pair should be balanced, so the expected count is half the pair sum;
3. Compute the statistic sum(observed - expected)^2 / expected over pairs with a positive sum;
4. Convert it to a p-value: a high p means "the data agree with uniform randomization" - suspicious;
5. The project repeats the statistic over 20 image prefixes and takes the median p for stability.

Watch the sign: here a **larger p is more suspicious**, because it says that fully randomized LSBs would easily produce what we observe - and a clean smooth image usually does not look like that.

> **Watch out | Natural noisy images fool the chi-square test.** Digital photos already have near-random LSBs, so p is naturally high. The project adds a content-randomness correction: it estimates how noisy the image is from difference entropy before treating a high p as evidence. This is a key engineering detail for turning a heuristic into a usable tool.

3.4 RS Analysis: Structural Collapse of the LSB Plane

Fridrich's RS analysis takes a different view: instead of comparing pair counts, it examines the **spatial structure** of the LSB plane. Group the pixel stream into groups of four, define a smoothness discriminant f (sum of absolute differences inside the group), then flip selected LSBs with a positive mask +M and a negative mask -M, and count groups where f grows (regular R) or shrinks (singular S).

Clean natural images show a strong regular tendency under the negative mask: Gn = (Rn - Sn) / N is clearly positive (project values for clean smooth images are often 0.3-0.7).

LSB replacement randomizes the plane and collapses this structure: Gn drops sharply. The project also outputs Gr and an estimated embedding rate.

Core RS logic from steganalysis.py (pseudocode; it illustrates grouping, not a runnable script)

```
groups = flat[:m].reshape(-1, 4)        # groups of 4 pixels
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1)   # original f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF        # +M flips
neg = (groups ^ np.array([1,0,0,1])) & 0xFF        # -M flips
Rm = (f(pos) > fg).sum();  Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum();  Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n          # clearly positive when clean, ~0 when random
```

3.5 The Project's Combined Verdict: analyze()

`analyze()` in `src/steganalysis.py` does not trust a single statistic. It combines four families of signals:

- Median chi-square p (were gray-level pairs flattened?);
- RS collapse (how far Gn fell below a content-adaptive baseline);
- Difference entropy and LSB difference entropy (is the randomness native to the image?);
- The sequence of prefix p-values (is randomization local or global?).

The three sensitivity modes (Strict / Balanced / Loose) move the possible/highly-likely thresholds and probability pull factors - in other words, they choose an operating point between false positives and missed detections.

3.6 Experiment: Hide an English Sentence, Then Catch It

Real project APIs: embed -> save -> analyze (safe to run step by step)

```
import sys; sys.path.insert(0, "src")
import numpy as np
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA
```

```
clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello nsF5!", method="nsF5", p=3, password="")
IO.save_image(stego, "output/lab3_stego.png")
print("changed pixels:", report["cover_changed"])
print("decoded:", extract_string(stego, method="nsF5", p=3))
for name, im in (("clean", clean), ("stego", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])
```

Try it | Re-run with a 5000-character message (see `src/test_steg.py`) and watch changed-pixel share, Gn drop, and probability rise. Then try a genuinely noisy photo and see whether chi-square and RS can still separate clean from stego.

3.7 Summary and Self-Check

- LSB replacement flattens gray-level pair counts (chi-square) and destroys LSB-plane structure (RS);
- A high p is not safety; check whether the image is naturally random first;
- False positives and missed detections trade off; the sensitivity setting picks the operating point;
- The project's analyze() combines statistical signals with a content baseline.

Think about it | Naive LSB hiding embeds 1 bit per pixel and changes half of all pixels on average. If you hide very little - under 1% of pixels - can chi-square still catch it? RS? Keep these questions in mind for Chapter 4.

Chapter 4 - Matrix Embedding and F5: Change Less, Hide More (Week 5)

> **Try it |** Goals: understand why embedding efficiency matters; work one p=3 Hamming syndrome-embedding example by hand; know where F5's shrinkage problem comes from.

4.1 From One Bit per Pixel to p Bits per Modification

Naive LSB costs about 0.5 pixel changes per bit. Suppose instead that every block of n pixels can carry p message bits while changing **at most one pixel**. Then the number of changes drops sharply. This idea is **matrix embedding**, and its mathematical engine is the binary Hamming code.

Embedding efficiency alpha = message bits carried per modification, with the theoretical value alpha(p) = p*2^p/(2^p - 1): about 2.67 for p=2, 3.43 for p=3, 4.27 for p=4. It grows with p, but so does the block length n = 2^p - 1.

> **Tip |** Why is larger p "cheaper"? A Hamming code uses n=2^p-1 positions to carry p bits. Larger p means less overhead per position, so one modification carries more information. The cost is bigger blocks and stricter carrier requirements.

4.2 Hamming Code Crash Course: Parity-Check Matrix and Syndrome

Let block length n = 2^p - 1 and build a p x n **parity-check matrix H** whose columns are all nonzero vectors of GF(2)^p. For p=3, the seven columns are the binary representations of 1..7. Collect the LSBs of a block into a column vector x and define the syndrome s = H*x (mod 2), a p-bit vector.

To embed p message bits m, we want to flip individual positions so the new syndrome equals m. Because every column of H is distinct, flipping position j changes the syndrome by s XOR H_j - so **any needed syndrome change corresponds to exactly one column**.

1. Compute the current syndrome s = H*x;
2. If s == m, this block needs no change;
3. Otherwise compute d = s XOR m (a nonzero p-bit vector);
4. Find the column j of H equal to d;
5. Flip the LSB of position j; now H*x' = m.

4.3 A Worked p=3 Example

Let the seven LSBs be x = [0,1,0,1,1,0,1]. With little-endian columns (LSB on top): H1=[1,0,0], H2=[0,1,0], H3=[1,1,0], H4=[0,0,1], and so on. The syndrome XORs the columns where x has 1 - positions 2, 4, 5, 7: H2 XOR H4 XOR H5 XOR H7 = [0,1,0] XOR [0,0,1] XOR [1,0,1] XOR [1,1,1] = [0,0,1], so s=[0,0,1].

Embed m=[1,1,0]: d = s XOR m = [0,0,1] XOR [1,1,0] = [1,1,1]. That matches H7, so flip the 7th LSB: x'=[0,1,0,1,1,0,0]. Check: H*x'=[1,1,0]=m. Done.

Seven pixels carry three bits, and on average only (2^3-1)/2^3 ~ 87.5% of blocks need a single flip - about 0.875 changes per block, or 3.43 bits per modification.

> **Try it |** `src/matrix_demo.py` turns this into a clickable teaching panel (the "matrix coding demo" tab in the GUI). Call `demo_step(p, x, m_bin)` and inspect the returned s, d, and the matched column, then verify H*x' == m yourself.

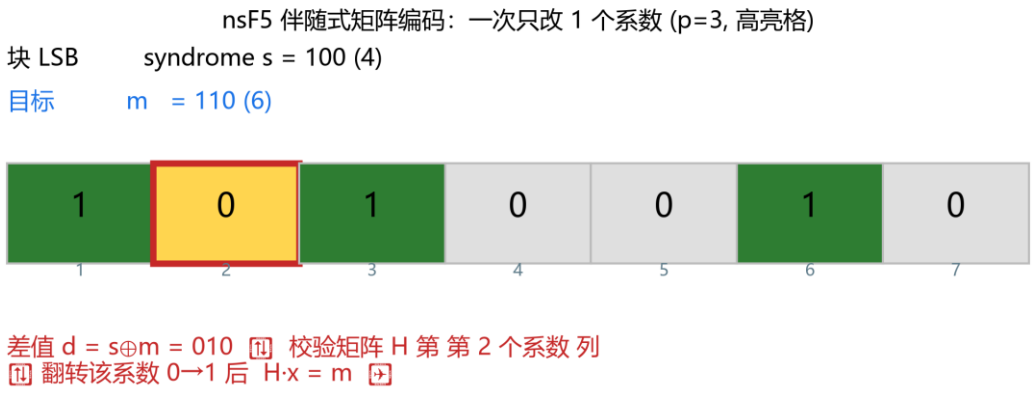

Fig. 4-1 Syndrome matrix embedding: at most one coefficient changed per block (thesis figure, p=3)

4.4 F5: Matrix Embedding on JPEG Coefficients

F5 (Westfeld, 2001) is JPEG steganography that applies matrix embedding to quantized DCT AC coefficients. Instead of flipping an LSB, it decreases the coefficient's **absolute value by 1** (e.g., +3 -> +2, -5 -> -4). That changes LSB parity without introducing large statistical anomalies.

The problem is **shrinkage**: when a coefficient with magnitude 1 is decreased, it becomes exactly 0. A zero coefficient means "no such frequency component" for JPEG, so F5 treats it as unusable, discards the block, and retries. Shrinkage lowers real payload, drops efficiency below the theoretical alpha, and leaves detectable statistical traces.

4.5 Back to the Project: MatrixEmbedding and Efficiency Curves

> **Read the code |** Open the `MatrixEmbedding` class near line 216 of `src/ns5_core.py`. Its `_embed()` is the direct code version of section 4.3: compute s, compare to m, find the column, flip. `src/efficiency.py` plots theoretical alpha(p) against the measured efficiency.

MatrixEmbedding._embed() core (excerpt of the real code)

```
xl = (c[pos] & 1).astype(np.uint8)     # take block LSBs
s  = syndrome(H, xl)                     # s = H*x
if np.array_equal(s, m): continue        # already matched
d = (s ^ m).astype(np.uint8)             # difference
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                         # flip the matched pixel
```

Generate the code-family and efficiency plot

```
import sys; sys.path.insert(0, "src")
from efficiency import plot_code_family_and_efficiency
path = plot_code_family_and_efficiency(p_max=6)
print(path)   # output/efficiency.png
```

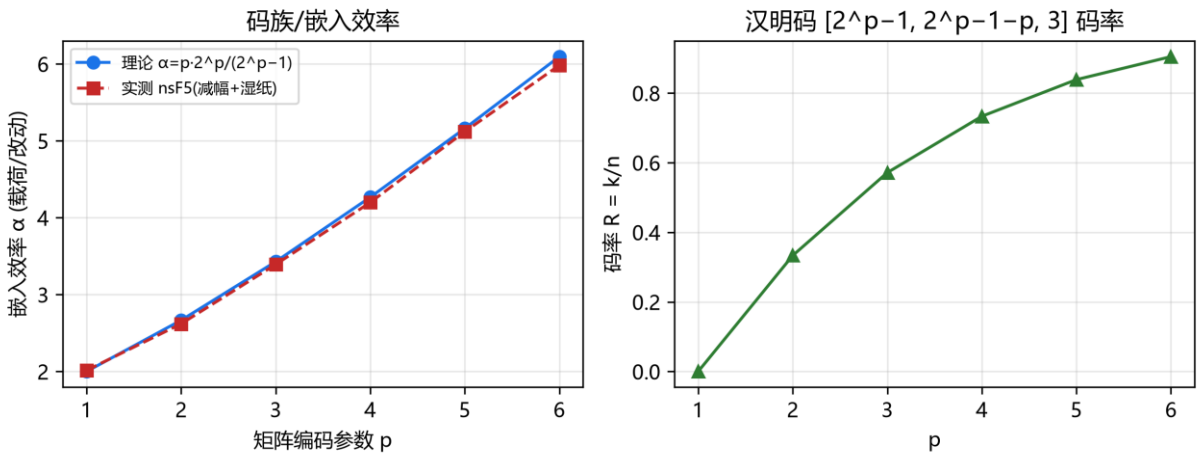

Fig. 4-2 Hamming code family and embedding efficiency: theory vs. measurement (thesis figure)

4.6 Summary and Self-Check

- Matrix embedding works on blocks: n positions carry p bits, at most one position changes;
- Distinct H columns mean every syndrome difference locates exactly one position to flip;
- Embedding efficiency alpha = p*2^p/(2^p-1), rising with p but so does block length;

- F5 decreases JPEG coefficient magnitudes and "shrinks" when magnitude 1 hits zero.

> **Think about it |** If F5 skips and retries a shrunk block, can the message still decode? How does the decoder know which blocks were skipped? If you cannot answer, that is fine - Chapter 5's nsF5 sidesteps the whole problem with wet paper coding.

Chapter 5 - nsF5 and Wet Paper Coding: The Project's Core Algorithm (Week 6)

> **Try it |** Goals: explain why shrinkage damages efficiency; understand wet paper coding as "wet positions untouched, solve on dry positions"; walk through the complete nsF5 flow in ns5_core.py.

5.1 State the Problem Clearly: F5 Shrinkage

F5 modifies JPEG quantized coefficients by decreasing magnitude, which flips the coefficient's LSB parity each time. But a coefficient with magnitude 1 (e.g., +1 or -1) becomes zero when decreased. For JPEG, zero means the band is absent, so F5 abandons that block and looks for another one. That is **shrinkage**.

- Shrinkage lowers **actual capacity**: blocks are wasted and the message needs more positions;
- Shrinkage lowers **embedding efficiency** below alpha(p), because lost blocks must be compensated;
- Shrinkage leaves detectable traces: too many coefficients were driven to zero, which shows up in histograms.

The project makes a clever spatial-domain analogy: subtract 128 from each pixel to get a signed value xv = pixel - 128. Pixels with |xv| <= 1 (that is, 127, 128, and 129) are the "danger zone" where decreasing the magnitude would land on or cross zero. The matrix-embedding shrinkage problem from JPEG becomes the same problem: those pixels cannot be ordinary modification targets.

5.2 Wet Paper Coding: Mark Wet Positions, Solve on Dry Ones

Wet paper coding, introduced by Fridrich and colleagues, answers one question: if some carrier positions cannot be touched (like ink on wet paper), can we still embed the full message using only the dry positions?

Yes - and the two parties do not need to agree in advance which positions are wet, as long as the algorithm can reconstruct the same wet/dry partition from the image content. The project does exactly this:

1. For each block, find positions whose magnitude decrease would hit zero/danger and mark them wet;
2. The real problem: find a modification vector e on the dry positions so the block syndrome becomes m;
3. That is, solve H[:, dry] * y = d over GF(2), where d = s XOR m;
4. The solver tries single dry-column hits, then pairs of dry columns, then a GF(2) Gaussian-elimination fallback, keeping changes sparse;
5. The solution y is a 0/1 vector; positions with y=1 are the dry positions actually modified.

> **Tip |** Why does pre-marking wet positions eliminate shrinkage? Shrinkage happened when a modification accidentally produced zero halfway through. nsF5 excludes all such positions in advance and solves only on safe dry positions, so every block embeds successfully: no retries, no surplus zeros.

5.3 Reading the Project: nsF5Pixel._embed()

In `src/ns5_core.py`, the `nsF5Pixel` class inherits from `MatrixEmbedding` and overrides only `_embed()`. Read that method line by line and you will see the complete nsF5 decision tree:

nsF5Pixel._embed() decision tree (pseudocode; logic matches the real code)

```
xv = c[pos].astype(np.int16) - 128     # pixel -> signed value
s = syndrome(H, (xv & 1).astype(np.uint8))
if s == m: continue                    # syndrome already matches
tc = matched_column(H, s ^ m)          # most direct candidate
if abs(xv[tc]) > 1:                    # safe to decrease
    c[pos[tc]] -= sign(xv[tc])         # one step toward 128
else:                                  # candidate is wet
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d)     # solve on dry positions
    if e:  decrease every dry position selected by e
    else:  fallback: flip the target LSB (keeps decodability)
```

5.4 The Three Solver Functions

Function	Purpose	Key point
solve_wet_paper(H, dry_cols, target)	Find a sparse solution on dry columns	Weight 1 -> weight 2 -> Gaussian fallback
gauss_solve_GF2(cols, b)	Solve a GF(2) linear system	Free variables set to 0 to minimize changes
permute_index(total, seed)	Deterministic pseudo-random permutation	splitmix64 + Fisher-Yates; C++ accelerated

Read the code | Open lines 145-215 of `src/ns5_core.py` and read the three functions against this table. Then run `test_wet_paper_needed()` in `src/test_core.py`: it forces pixels to 127/128/129 to create wet positions and still completes the embed/decode round trip.

5.5 Verification: Round Trips and Efficiency

Run the core self-test and compare the two methods

```
python src\test_core.py
# compare matrix vs nsF5 changed pixels:
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
```

```
    stego, rep, nb = embed_string(img, "Hello nsF5!", method=method, p=3)
    print(method, "changed", rep["cover_changed"], "pixels / capacity", nb)
```

On the same random image and message, matrix and nsF5 may change similar numbers of pixels, because danger-zone pixels (127/128/129) are rare in random data. The real difference appears when danger-zone pixels are dense: nsF5 still embeds without shrinkage while matrix embedding keeps failing.

5.6 Summary and Self-Check

- Shrinkage = a magnitude decrease hits zero, ruining the block, capacity, efficiency, and statistics;
- Wet paper coding marks untouchable positions as wet and solves the syndrome equation only on dry ones;
- nsF5 = matrix embedding + wet paper coding: no shrinkage, no retries;
- In the pixel domain the project uses xv = pixel - 128 and treats |xv| <= 1 as wet.

> **Think about it |** Why does wet paper coding not require the decoder to know the wet positions? Hint: the decoder only reads LSBs and computes syndromes; it never needs to know which positions the embedder skipped.

Chapter 6 - Passwords, SHA-256, and Keyed Security Design (Weeks 6-7)

> **Try it |** Goals: understand why embedding positions must depend on a key; explain decoder self-synchronization and tamper perception; know the limits of this design.

6.1 What Happens When Positions Are Fixed?

Suppose embedding always starts at the top-left corner and moves in order. An attacker who knows the algorithm can: (1) read the LSBs of the first N pixels and recover the message; (2) copy the message to another "identical-looking" image (position-replacement attack); (3) perturb only the fixed region to destroy your message without touching the rest of the image.

Modern steganography therefore scrambles the embedding path with a **key-controlled pseudo-random order**. Without the password (or content hash), nobody knows which block holds the message and position-level attacks fail.

6.2 Two Seed Rules: Recognize the Image First, Then Find the Payload

The project splits pixels into a **header pool** (the first N_h blocks) and a **payload pool** (the remaining pixels). Two independent seeds are derived:

- **seed0 = derived from the password only**: scrambles the header pool. The header stores cover_hash - the first 16 bytes (128 bits) of the SHA-256 of the original image bytes;
- **seedB = derived from cover_hash + password**: scrambles the payload pool and block ordering.

To decode, the receiver rebuilds seed0 from the password, reads the header to recover cover_hash, and only then derives seedB to find the payload. A wrong password or a broken header kills the path immediately - that is what `test_pwd_mismatch()` verifies.

Seed derivation in the project (real code)

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
            base.encode() + password.encode()).hexdigest()
    return int(base, 16)
```

6.3 Deterministic Shuffling: splitmix64 + Fisher-Yates

A seed alone is not enough; it must expand into a permutation of 1..N. The project uses splitmix64 as the pseudo-random source and performs Fisher-Yates: walk from the end to the front, each step swapping with a random earlier position. The same seed always gives the same permutation - the mathematical basis of self-synchronization.

> **Read the code |** Open `permute_index()` in `src/ns5_core.py`: with the DLL present it calls C++ `nsf5_permute`; otherwise it runs an identical Python fallback. **Both implementations must produce exactly the same permutation**, otherwise an image embedded with the DLL cannot be decoded on a machine without it. `selfcheck()` in `src/cppembed.py` verifies cross-language consistency.

6.4 Tamper Perception: What It Can and Cannot Do

The embedder reports `cover_hash` (the original image hash). Any pixel change alters the SHA-256 of the image bytes. Recompute the hash of the received image and compare: mismatch means the image was touched. More elegantly, because the payload path depends on that hash, even a one-pixel change shifts the payload seed and the decode collapses or produces garbage.

Be honest about the boundary: this is **keying plus tamper perception**, not cryptographic **integrity authentication**. The header hash is not keyed with a MAC, so an attacker capable of re-embedding the whole image can update the header hash too. The README and thesis list a keyed MAC as future work; keep the distinction between "detect ordinary tampering" and "resist a malicious re-embedder" clear.

6.5 Experiments: Wrong Password and One-Pixel Tampering

Experiment A: wrong password must fail; Experiment B: tamper one pixel

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
from ns5_core import embed_string, extract_string, get_image_hash


img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1   # flip one pixel
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```

> **Watch out |** Experiment B may fail as a ValueError, garbage text, or a truncated message, depending on which block the pixel belongs to. Do not expect an exception every time: the engineering reality is "hash mismatch plus failed self-synchronization," not a clean error message.

6.6 Summary and Self-Check

- Fixed paths are predictable: readable, copyable, attackable; keyed paths make positions depend on password + content;
- The header stores a 128-bit content hash and the payload seed depends on it - decoding needs no external sync;

- Deterministic permutation (splitmix64 + Fisher-Yates) is the mathematical guarantee of self-synchronization;
- Keying is not integrity authentication; strong adversaries require a keyed MAC.

> **Think about it |** If two different cover images hide the same message, will their stego images use the same hiding positions? Why or why not? (Hint: what does the payload seed depend on?)

Chapter 7 - Machine Learning Foundations (First Time Learners, Weeks 7-8)

> **Try it |** Goals: build the full data -> feature -> model -> evaluation frame; explain overfitting, cross-validation, and data leakage; read ROC and AUC correctly.

7.1 Steganalysis Is a Binary Classification Problem

Change perspective: for any image, it is either a clean carrier (label 0) or a carrier that hid a message (label 1). Machine learning learns a function f(image) -> 0/1, or "stego probability." That is exactly what the project's supervised steganalysis does.

Supervised learning has four pieces: **data, features, model, evaluation**. Each concept below maps to project code immediately.

7.2 Data: Samples, Features, Labels

Version 1 compresses each image into 11 numbers called a **feature vector** and records its class (**label**). Version 1.4 adds a 143-D v2 feature set on top (Section 8.8). For teaching, start with the v1 table `data/dataset.csv`: 2,898 rows = 414 photos x (1 clean + 6 stego variants), of which 414 are clean and 2,484 are stego.

data/dataset.csv (real first two rows, truncated for display)

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,
lsb_diff_entropy,median_prefix_p,chi2_stat,label,photo_id,...
41316,4574,36608,5057,0.56,0.48,3.5e-278,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,2.0e-269,2.02,0.989,2.2e-155,1675.9,1,0,...
```

Notice `photo_id`: the clean version and all variants of one photo share an id. That unremarkable column is the key to preventing data leakage in 7.4 (v1.4's v2 datasets add more variants and real JPEG clean samples, but the grouping idea is unchanged).

7.3 How a Model Learns: Loss, Gradient, Logistic Regression

Start with the most explainable model - **logistic regression**: a linear combination z = w*x + b passes through the sigmoid sigma(z)=1/(1+e^-z), producing a 0-1 probability.

Training means finding weights w that minimize the **loss** over all samples. Logistic regression typically uses cross-entropy: predicting 1 when the truth is 0 is punished, and the more confident the wrong prediction, the steeper the penalty. Optimizers such as gradient descent nudge w downhill until the loss stops falling.

> **Tip |** You do not need to derive gradients now. Remember three sentences: **loss = how wrong the model is; training = find parameters that minimize loss; gradient descent = keep moving in the steepest downhill direction.** The project wraps all this in scikit-learn's LogisticRegression.

7.4 Overfitting, Cross-Validation, and Data Leakage

Performing well on data the model has already seen is easy. The real test is **unseen data** - that is generalization. A model that memorizes the training samples collapses on the test set: **overfitting**.

The standard defense is **cross-validation**: split the data into folds and rotate which fold is the validation set. But there is a subtle trap: all seven samples from one photo are highly similar. A random split can put a "sibling sample" of a test photo into training, which means the model has effectively peeked at the test photo - **data leakage** - and the AUC is unrealistically high.

> **Watch out |** The project uses `GroupKFold`: splits are grouped by photo id, so every variant of a photo is either entirely in training or entirely in validation. That single choice in `cv_auc()` (src/train_model.py) decides whether the experiment is trustworthy. It is one of the most valuable machine-learning engineering lessons in this project.

7.5 Evaluation Metrics: Accuracy Is Not Enough

Stego detection has imbalanced classes and asymmetric costs (flagging a clean image differs from missing a stego image), so one number is not enough:

Metric	Question it answers	Intuition
Accuracy	Overall fraction correct	Misleading when classes are imbalanced
Precision	Of flagged stego, how many truly are	Low false positives -> high precision
Recall	Of true stego, how many were caught	Low misses -> high recall
F1	Harmonic mean of precision and recall	Compromise when you want both
ROC / AUC	Ranking ability independent of threshold	AUC 0.5 = random guessing

Models output probabilities, so a **threshold** must turn them into 0/1 decisions. Higher thresholds are conservative (fewer false positives, more misses); lower thresholds are aggressive. Training picks a **Youden** threshold where true-positive rate minus false-positive rate is largest, and also computes a separate low-false-positive threshold (FP <= 10%) for the Strict mode.

Youden threshold (real logic)

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                       # Youden J
best = float(th[np.argmax(j)])      # point farthest from random
```

> **Try it |** After this section, run `python src\train_model.py` once. You do not need every line of output; find three: CV-AUC, held-out test AUC, and low-FP detection rate. Chapter 8 explains them line by line.

7.6 Summary and Self-Check

- Supervised learning = learn a function from features to answers using labeled samples;
- Random splits leak information between same-source samples; GroupKFold splits are the honest option;
- AUC measures ranking; the threshold decides the operating point;
- False-positive rate and detection rate trade off; Youden and low-FP thresholds are two choices.

> **Think about it |** A paper reports steganalysis AUC 0.95 but never says how data were split. What do you suspect first? Write it down - that instinct is the start of critical ML evaluation.

Chapter 8 - Machine-Learning Steganalysis (Weeks 8-9)

> **Try it |** Goals: understand why the project uses 11-D statistical features instead of raw-pixel CNNs; know what each feature measures; run the data -> train -> evaluate -> GUI pipeline; explain the v1.4 SRM / 143-D / dual-model story.

8.1 The Counterintuitive Question: Why Not a Raw-Pixel CNN?

Feeding raw pixels to a deep CNN does not work well here. The project tried multiple architectures on 414 photos, and validation AUC hovered around 0.50 - no generalization at all. Why?

- Steganographic changes are an extremely weak high-frequency signal (LSB layer), far below objects and textures;
- A CNN easily learns photo content rather than hiding traces, and fails as soon as test photos look different;
- The number of truly independent samples is the number of photos (hundreds), while deep models usually need thousands of independent sources.

The project therefore uses **domain knowledge as features plus simple classifiers**: hand-design features that directly measure LSB statistical footprints. On the same small dataset, the v1 11-D feature route reached AUC ~0.75-0.79, and every feature is explainable. v1.4 raised this to 0.90+ with 143-D features and LightGBM (Section 8.8), while the "features beat raw CNN" conclusion stands.

> **Tip |** This is not a "deep learning is useless" claim. It is a lesson in modeling under sample constraints: first verify the signal with strong explainable features, then consider more complex models. Many engineering problems follow the same order.

8.2 Meet the v1 11-D Features

The v1 features are all computed by the C++ library `cpp/fsfeatures.dll` (Python binding in `src/fsfeatures.py`); the 143-D v2 set is assembled in `src/featurize_v2.py`. The 11 features fall into three groups:

Feature	Meaning	After embedding
Rm / Sm / Gr	Regular/singular group counts and gap under +M	Gr drops (structure damaged)
Rn / Sn / Gn	Counts and gap under -M	Gn collapses (core signal)
chi2_stat / chi2_pvalue	Chi-square statistic and p-value	Gray pairs balance, p rises
diff_entropy	Difference entropy: native texture roughness	Used to identify naturally noisy images
lsb_diff_entropy	LSB-plane difference entropy	Approaches 1 as LSBs randomize
median_prefix_p	Median chi-square p over 20 prefixes	Rises with global randomization

> **Read the code |** Open `src/steganalysis.py` and find `diff_entropy()` and `lsb_diff_entropy()`: they estimate the randomness baseline with Shannon entropy of adjacent differences. Then open `src/fsfeatures.py` to see how the feature order is passed to C++.

8.3 Where Data Come From: The "1 + 6" Design of make_dataset.py

Training data cannot come from a few hand-hid messages. `src/make_dataset.py` produces one clean image plus six stego variants per source photo (v1.4 also supports 12 v2 variants; see 8.8):

Variant (method, p, density)	Strength	Purpose
clean	-	Provides clean negative samples

Variant (method, p, density)	Strength	Purpose
nsF5, p=3, d=0.25	weak	Close to realistic covert communication
nsF5, p=3, d=0.55	medium	Moderate strength
nsF5, p=2, d=0.35	weak-medium	Lower p is less efficient, leaves more traces
nsF5, p=2, d=0.85	strong	Heavy embedding, obvious footprint
matrix, p=3, d=0.50	medium	Compare a second method
matrix, p=2, d=0.80	strong	High-density plain matrix embedding
v1.4 adds +6	-	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

All images are resized to 512x512 grayscale; features come from C++ and stego variants are produced by the C++ embedding path, so training and inference stay consistent.

8.4 How to Read the Training Script: train_model.py

`python src\train_model.py` is one honest evaluation pipeline:

1. Load the table and drop rows with missing features;
2. Reserve 25% as a held-out test set with GroupShuffleSplit by photo_id (those photos never enter training);
3. Run 5-fold GroupKFold on the training pool, comparing LogisticRegression / RandomForest / GradientBoosting / XGBoost (v1.4 adds LightGBM tuning and stacking);
4. Pick the best CV model, carve 20% of the training pool as a calibration set, and choose Youden plus low-FP thresholds;
5. Retrain on the full training pool and report accuracy, balanced accuracy, AUC, and classification report on held-out data;

6. Report per-(method, p, density) detection rates;
7. Save model and thresholds (v1.4 keeps dual versions; see 8.8).

> **Watch out |** The calibration set is for threshold selection and the test set is used exactly once. Re-tuning thresholds on the test set dirties it. Training/calibration/test is the same three-layer discipline industry uses.

8.5 Reading Results: ROC, AUC, and Per-Density Rates

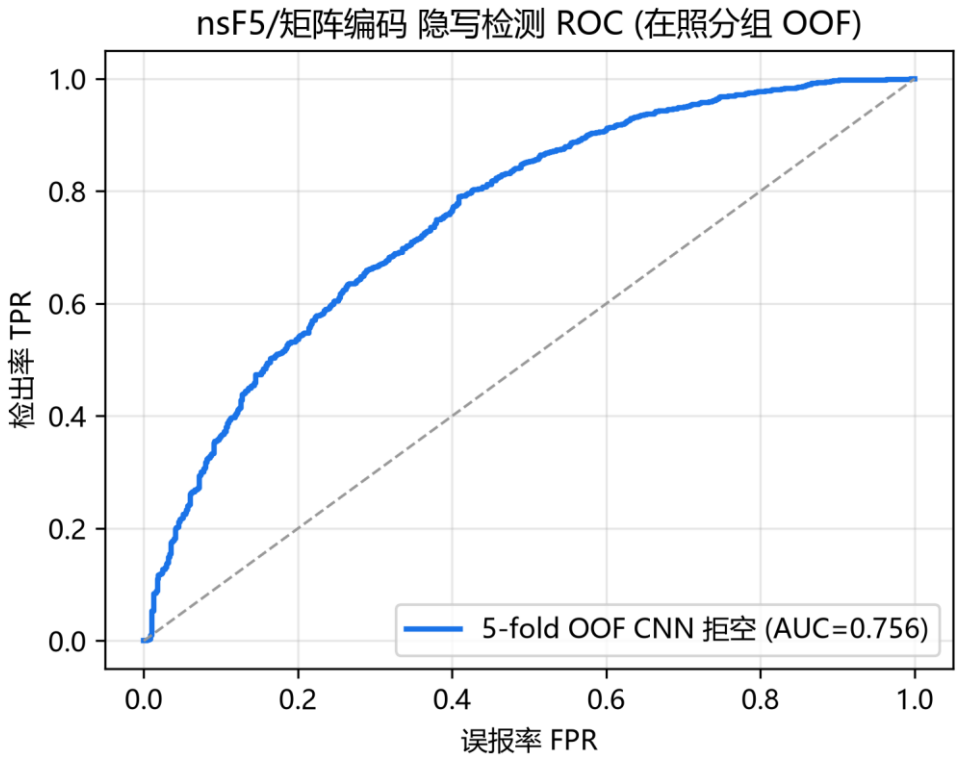

Fig. 8-1 Steganalysis ROC (photo-grouped held-out folds; v1 11-D baseline, AUC ~0.756)

The ROC curve sweeps the threshold from 1 to 0 and plots detection rate against false-positive rate. The closer to the top-left corner, the better. AUC is the area under it: 0.5 is random. The v1 baseline around 0.75 means "usually ranks stego above clean," but weak

densities still escape. The v1.4 dual models raise the 8-split average AUC to 0.9085-0.9227 (Section 8.8).

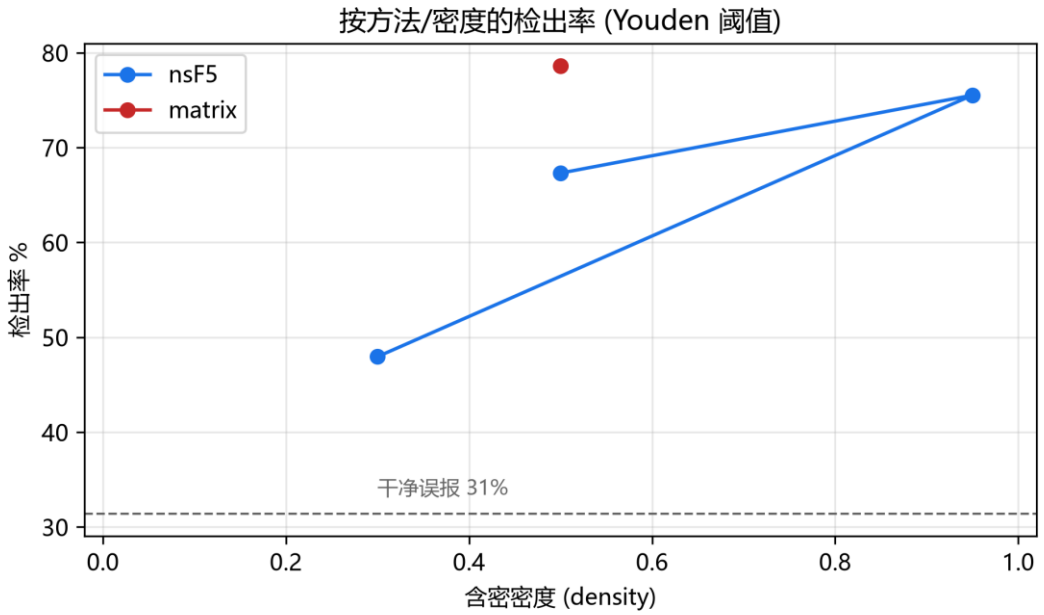

Fig. 8-2 Per-method/per-density detection (Youden threshold; v1 6-variant baseline): strong densities high, weak densities drop

> **Read the code |** Compare Fig. 8-2 with `thesis/data/det_density.csv`: the weak nsF5 p3 band lands around 48-64%, while strong matrix p2/p3 reaches 75-99%. Weak embedding being harder to detect is not mysticism; it is the direct result of a smaller statistical footprint.

8.6 Sensitivity in the GUI and in ml_predict.py

`MLPredictor` in `src/ml_predict.py` preprocesses the image (grayscale -> 512x512), then automatically picks the v1 11-D or v2 143-D feature path based on the model's stored feature list, and outputs a stego probability. Sensitivity works by switching thresholds:

Sensitivity	ML threshold	Effect
Strict (low FP)	threshold_low_fp	Fewest clean-image errors; more weak stego missed

Sensitivity	ML threshold	Effect
Balanced	Youden threshold	Balanced detection vs. false positives; default
Loose (high detection)	Youden x 0.75	Catches more weak densities; false positives rise

The GUI shows heuristic and ML probabilities side by side and asks you to **cross-check**, not trust one number - a deliberate honesty feature of the project.

8.7 Experiments: From Training to Single-Image Prediction

The full pipeline (minutes to tens of minutes, machine dependent)

```
# 1) generate a dataset (point it at your own photo folder)
python src\make_dataset.py data\campus_jpg --out campus


# 2) train and evaluate
python src\train_model.py


# 3) single-image ML prediction
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
s, _, _ = embed_string(a, "ML test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

> **Try it |** Run an honesty experiment: predict on a clean photo (should be near clean), then embed a weak-band message (nsF5 p=3 near density 0.25) and predict again. The probability may move only slightly - that is the README warning about weak densities, live.

8.8 v1.4.0 Update: SRM, 143-D Features, and Dual Models (2026-09)

Sections 8.1-8.7 describe the v1.0-v1.3 route: 11-D features with LR/XGB and AUC around 0.75-0.79. v1.4.0 upgraded three things: SRM high-pass preprocessing, a 143-D v2 feature set with 12 variants, and finally a dual LightGBM model strategy. The story

below follows the project's actual order so that the README's long experiment tables make sense.

8.8.1 SRM High-Pass Filtering: Same-Source Gain, Cross-Source Loss

SRM (Spatial Rich Model) is a family of high-pass filters from the Fridrich framework. `src/srm_filter.py` implements the standard 30 kernels (8 first-order, 4 second-order, 8 third-order differences, 8 edge kernels, 2 square kernels), filters the image, takes the maximum absolute response per pixel, clips to +/-4, and rescales to a single uint8 "enhanced image" that feeds the existing feature extractors. CPU (numpy) and GPU (torch) implementations are exposed through `make_dataset.py --preprocess srm` and the GPU `use_srm=True` flag.

Metric	Same-source baseline	Same-source + SRM
5-fold CV-AUC	0.7594	0.7922
Held-out test AUC	0.7811	0.8085
Weak nsF5 p3 d0.25 detection	51.9%	62.5%
nsF5 p2 d0.35 detection	76.0%	90.4%
Low-FP stego detection	41.2%	50.3%
Clean false positives (low-FP thr)	9.6%	9.6%

> **Watch out |** The gain is only within the same source domain. In cross-source training (campus + full BOSSbase), SRM pulls CPU test AUC from 0.741 down to 0.704 and GPU validation AUC from 0.651 down to 0.572. High-pass filtering also flattens the signals that distinguish different cameras/compression sources. That is why the default model is trained on merged data without SRM, while the SRM campus model is kept as a separate same-source artifact.

8.8.2 The v2 143-D Feature Set and 12 Variants

`src/featurize_v2.py` adds to the 11 base features: mean / absolute mean / standard deviation over 30 SRM residuals (90 D), 20 full-image prefix chi-square p-values, 20 LSB-prefix p-values, plus texture_noise and est_rate (2 D): 11 + 90 + 20 + 2 + 20 = 143. The training variants grow from 6 to 12: nsF5 p3 d0.40, matrix p3 d0.40/0.60, and lsb d0.30/0.50/0.70 are added.

Strict A/B results (same test photo groups, 5-fold GroupKFold OOF): the 12-variant v2 dataset alone adds +0.019 (0.7868 vs 0.7678 at equal 11-D features); adding the 143-D set adds another +0.027 (OOF 0.8143, held-out 0.8278). Combined, about +0.05 AUC.

8.8.3 Real JPEG Clean Images Expose a Deployment Problem - and the Fix

The v2 logistic-regression model scored well inside its training distribution but flagged almost every real JPEG clean photo as stego with probability 1.0: SRM residuals react strongly to JPEG high-frequency noise, and clean/stego logits overlap heavily. The fix was to add 414 real JPEG clean photos as independent photo_ids (fully disjoint from the original training groups), producing dataset_campus_v2_jpeg.csv, and then grid-tune LightGBM. The tuned LGB became the new default: held-out AUC 0.8946, 8-split average 0.9085, and weak nsF5 p3 d0.25 detection rose from 67.9% to 85.3%.

8.8.4 Dual-version Deployment: 143d Robust vs. 53d Interpretable

Interpretability experiments showed that ~73% of the gain in the 143-D set comes from 31 explainable features (BASE 11 + PREFIX 20); the SRM 90 dimensions average single-feature AUC 0.50-0.52 (near random) yet act as a buffer that filters JPEG noise and improves out-of-distribution robustness. The project therefore ships both:

Dimension	143d default (robust)	53d interpretable
Features	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	Same minus SRM90 (53 D)

Dimension	143d default (robust)	53d interpretable
8-split mean AUC	0.9085	0.9227
Held-out AUC	0.8946	0.9100
Weak nsF5 p3 d0.25	85.3%	87.2%
Real-JPEG clean OOD	1/8 FP (median 0.011)	3/8 FP (median 0.028)
Best for	Deployment / mixed domains / real images	Papers / defenses / teaching / per-image explanations

`MLPredictor()` loads the 143d default; pass `model_path` to switch to the 53d model. Prediction automatically selects 11-D or 143-D features from the model metadata. GUI model switching is still planned; for now, switch through the API:

Switching between the dual models

```
from ml_predict import MLPredictor


# default: 143d robust model (models/stego_classifier.joblib)
pred_143 = MLPredictor()


# 53d interpretable model (SRM 90 removed; higher AUC)
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)


r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

Try it | Experiment: train on dataset_campus_v2_jpeg.csv (set $env:DS_FILES), then compare 143d vs 53d on real JPEG clean photos and on weak nsF5 p3 d0.25 images. Put the AUC and OOD behavior into your report.

Watch out | The v2/53d models are only reliable near their training distribution: campus-derived PGM/BMP grayscale plus the added JPEG clean photos. For a brand-new camera or

compression source, run a small OOD test before deployment.

8.9 Summary and Self-Check

- Under small samples and weak signals, domain features + simple models usually beat raw-pixel CNNs;
- The 11 v1 features = RS family + chi-square family + entropy/randomness baselines;
- Training/calibration/test separation plus GroupKFold is the skeleton of experimental trust;
- ML probability should be cross-checked with heuristics; weak densities have physical limits;
- Since v1.4, dual models are strong inside their distribution, but real-JPEG OOD validation remains mandatory.

Think about it | Seven samples come from one photo. Why is a random 80/20 split not acceptable? Design a minimal example showing how much leakage can inflate AUC.

Chapter 9 - Engineering: C++, GPU, GUI, and Tests (Week 10)

> **Try it |** Goals: see how algorithms become software; understand why C++ and GPU acceleration exist and how cross-language consistency is guaranteed; run the tests and trace the CI.

9.1 Architecture: Layered, Not Coupled

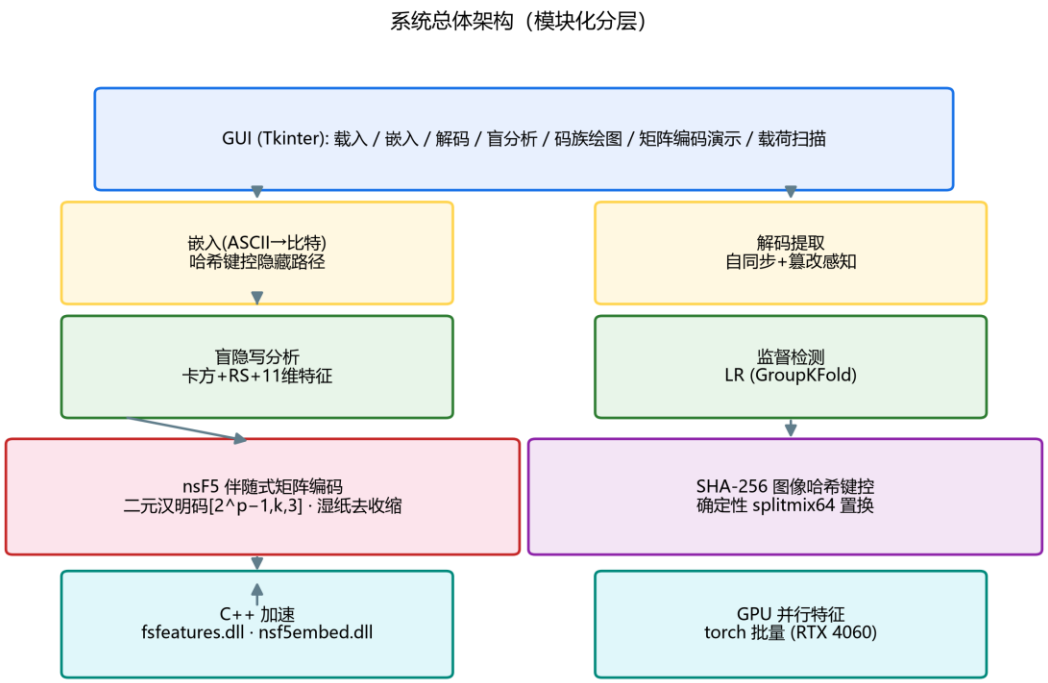

Fig. 9-1 System architecture: acceleration layer -> algorithm/security core -> features -> GUI (thesis figure)

- **Acceleration layer:** cpp/fsfeatures.dll (features), cpp/nsf5embed.dll (embedding/shuffling);
- **Algorithm core:** ns5_core.py - Hamming codes, wet paper, hash keying, embed/extract;

- **Feature layer:** steganalysis.py, efficiency.py, make_dataset.py, train_model.py, ml_predict.py, featurize_v2.py, srm_filter.py;
- **UI layer:** gui.py + matrix_demo.py + scan_panel.py - one-click access to everything.

Layering matters because the algorithm layer does not depend on the GUI (it runs on servers and in CI), and the acceleration layer falls back to pure Python when a DLL is missing, so the features always work.

9.2 C++ Acceleration: Hot Paths and How We Know They Are Right

Python is slow at elementwise loops, and this project has two classic hot paths: **deterministic permutation** (shuffling up to 16 million positions) and **feature extraction** (RS/chi-square/entropy statistics per image). Moving them into C++ DLLs gives orders-of-magnitude gains: permutation speedup up to ~263x (4096^2 image: ~12.3 s in Python -> ~223 ms in C++).

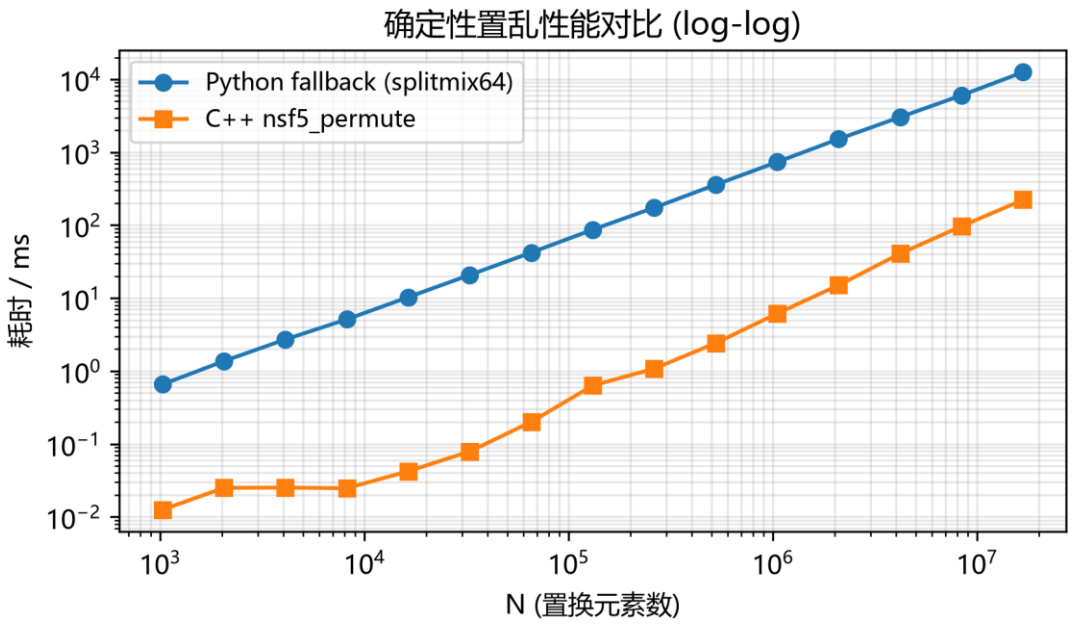

Fig. 9-2 Deterministic permutation: Python vs C++ (log-log axes, thesis figure)

Speed is worthless unless the result is identical. Python calls the DLLs through ctypes, and the project guards correctness with self-checks:

- `cppembed.py` embeds the same image through C++ and Python paths and compares outputs pixel by pixel;
- `fsfeatures.py` check_against_python() compares every C++ feature with the Python reference;
- When the DLL is missing or fails to load, the code falls back to the same Python algorithm, keeping embed/decode reversible everywhere.

> **Watch out |** Cross-language consistency is the prerequisite for acceleration and a classic debugging minefield: bitwise operations, rounding, and overflow differ between C++ and NumPy. If you ever see "embeds with DLL, cannot decode without it," run `cppembed.selfcheck()` and `fsfeatures.check_against_python()` before touching the algorithm.

9.3 GPU Version: Batch-Vectorizing Features

The `gpu/` directory rewrites feature computation as PyTorch tensor operators: load a batch of 512x512 grayscale images, compute histograms, RS, chi-square, and entropies in parallel on the GPU, then return results to the CPU. v1.4 adds `gpu/featurize_v2_gpu.py` for the 143-D v2 features (including SRM convolutions). The v1 pipeline extracts 2,070 images in about 5 seconds (~410 images/s) and matches the CPU reference bit-for-bit (float error ~1e-7).

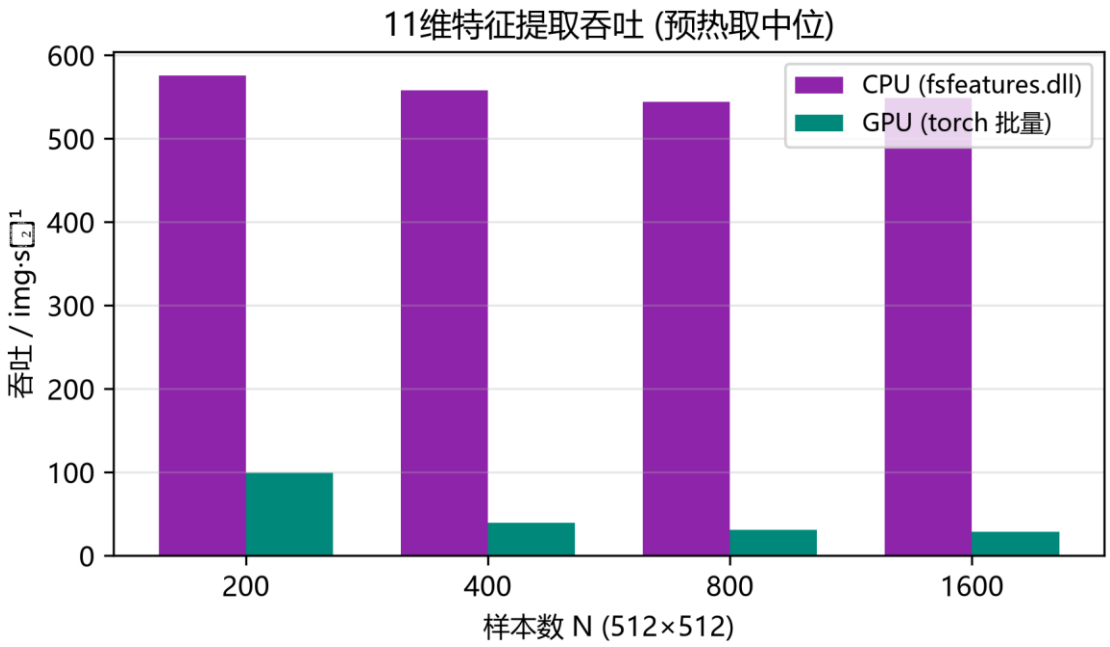

Fig. 9-3 v1 11-D feature throughput: CPU (C++) vs GPU (torch batch) (thesis figure)

> **Read the code |** Read the self-check in `gpu/featurize_gpu.py`: it compares GPU results element by element with `src/fsfeatures.py`. Bit-level consistency is what makes the dual implementation safe. Then read the README discussion of throughput limits: for small batches, host-device transfer can make GPU slower than C++ - an honest measurement, not a marketing claim.

9.4 GUI: Turning a Research Tool into a Teaching Application

`src/gui.py` is organized around Embed -> Decode -> Analyze. Two teaching panels stand out:

- **Matrix coding demo** (matrix_demo.py): click a block, watch s, m, and d update live, see the matched H column highlighted, and verify H*x' = m after the flip;

- **Payload scan** (scan_panel.py): drag payload from 0 to 0.4 and watch chi-square p, RS estimate, and ML probability curves refresh as the image is re-embedded at each density.

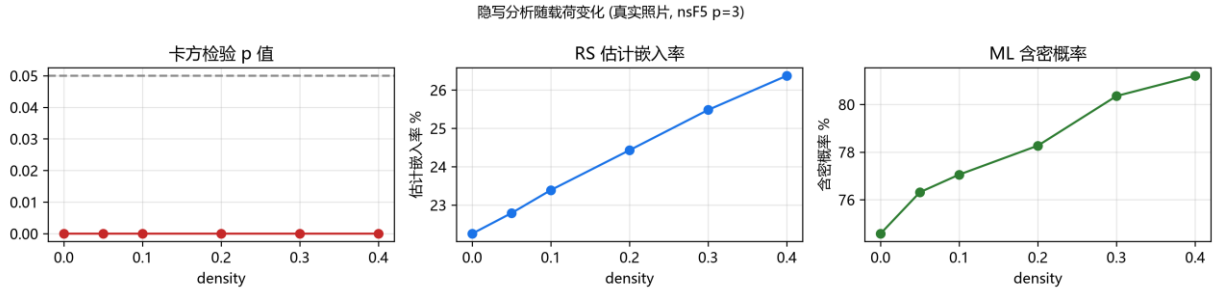

Fig. 9-4 Payload scan: detectability grows with embedding density (thesis figure)

> **Watch out |** Note the wording in the GUI: a single image has no true "AUC" (AUC needs a set of positive and negative samples), so the scan panel plots model probability as a trend illustration. Distinguish "single-image probability" from "dataset metrics" whenever you read software output.

9.5 Tests and CI: Refactor Without Regret

Test file	Verifies	Mental model
test_core.py	Hamming matrices, embed/decode round trips, wet paper, wrong passwords	Algorithms still work
test_steg.py	Blind analysis separates clean from stego	Analysis still works
test_false_positive.py	Clean images are not flagged (regression)	False positives stay controlled
test_gui.py	Window builds, loads images, previews	UI still works

`.github/workflows/ci.yml` runs core tests and builds wheel + sdist on every push/PR to main; pushing a `v*` tag creates a GitHub Release. Tests + packaging + auto-release is the standard way an algorithm project becomes a reusable tool.

9.6 A Code-Reading Route (Use as Needed)

1. Entry point: src/run_e2e.py - the shortest path through every module;
2. Data layer: image_io.py - how arrays enter and leave;
3. Embedding layer: read ns5_core.py top to bottom (hash, shuffle, Hamming, wet paper, high-level API);
4. Analysis layer: trace analyze() in steganalysis.py backwards through each statistic;
5. ML layer: train_model.py main flow -> featurize_v2.py / srm_filter.py (v1.4) -> ml_predict.py dual-model inference;
6. Acceleration layer: cppembed.py / fsfeatures.py ctypes bindings and self-checks;
7. UI layer: find a button callback in gui.py and follow it to the algorithm function.

> **Think about it |** Why does the comment in ns5_core.py insist that changing the permutation algorithm breaks reversibility? Combine with the v1.2.1 fix (automatic Python fallback): what catastrophe happens if Python and C++ permutations differ?

9.7 v1.4.0 Update: SRM, Multi-Source Data, and Model Engineering (2026-09)

Several engineering modules changed in v1.4.0. Learn the new directory first:

New file / module	Role	What to study
src/srm_filter.py	30 standard SRM high-pass kernels; numpy/torch dual implementations	Kernel list, normalization, enhanced-image pipeline
src/featurize_v2.py	143-D v2 feature assembly on CPU	ALL_FEATURE_NAMES, featurize_v2()
gpu/featurize_v2_gpu.py	GPU batch 143-D features and consistency self-check	extract_features_v2_gpu()

New file / module	Role	What to study
src/make_dataset.py extensions	Multiprocessing -j, SRM/feature-set/variants options	Per-image parallelism, identical row output
src/train_model.py extensions	DS_FILES multi-dataset, stacking, LGB tuning	Dual-model saving and thresholds
gpu/make_imageset.py extensions	memmap disk write, --out/--id-offset	Multi-source partitioning without OOM

On the data side, v1.4 established a pure campus-photo baseline `data/campus_jpg` (414 photos; earlier DIP4E textbook images were removed) and added the standard steganalysis benchmark BOSSbase 1.01 (10,000 512x512 grayscale PGM images) for multi-source training. CPU merged training used 72,898 samples (held-out AUC ~0.741); GPU merged training used 52,070 samples (validation AUC ~0.712); BOSSbase alone reached only ~0.644 on GPU - a harder benchmark with weaker signals. `make_dataset.py` now parallelizes across images (~5.6x speedup on 16 cores), and GPU image sets use memmap writes to avoid OOM.

Tooling changed too: the license moved from MIT to Apache-2.0 with a NOTICE file and third-party attributions, and README/pyproject now declare v1.4.0. The pixel-level and feature-level self-checks between C++ and Python remain in place.

> **Try it |** Run `python gpu\featurize_v2_gpu.py` self-check. Then follow the README to generate one source from `data\campus_jpg` and one from `data\BOSSbase_1.01`, and compare three models: campus only, BOSSbase only, and merged. That is the most instructive data-domain experiment in v1.4.

Chapter 10 - Capstone: From Reading to Building (Weeks 11-12)

> **Try it |** Goals: complete one demonstrable improvement experiment in two weeks. Three directions are given below, easy to hard; you may combine them.

10.1 Direction A: Reproduce and Critically Verify (Safest)

Choose one or two claims from the README/thesis, rerun them, and try to break them:

- Reproduce the code-family figure: does measured alpha(p) match theory? Where does deviation creep in at large p?
- Reproduce detection with your own photos: compare the v1 11-D baseline (~0.75-0.79) with the README's v1.4 dual models, and see how photo style moves AUC;
- Reproduce the tamper experiment: flip 1 pixel, 1 row, 1 block - what are the failure rates?

Acceptance item	Passing standard
Experiment log	Environment, commands, parameters, raw outputs recorded
Figures	At least two plots with complete axes and legends
Conclusions	Distinguish "confirms the paper" from "disagrees with the paper" with reasons
Reproducibility	One-click script or a list of commands others can rerun

10.2 Direction B: Feature Extensions (Most Satisfying)

- **UTF-8 / Chinese messages:** change encode_string() from ASCII to UTF-8 (remember the length header counts bytes, not characters);

- **Color channels:** only the R channel is used today; study how to distribute capacity and security across R/G/B;
- **New features:** add a statistic you believe separates clean/stego on top of v1 11-D or v2 143-D, and measure with GroupKFold whether AUC really rises;
- **Baseline comparisons:** implement naive LSB replacement and compare Gn collapse and detection rates against nsF5 at equal payload.

> **Watch out |** After every extension, run test_core.py and test_steg.py: embed -> decode must still round trip. When you change the encoding scheme, the decoder must change with it - the most commonly forgotten step.

10.3 Direction C: Repackage a Teaching Demo

Turn the GUI matrix-coding demo into a self-contained lesson or animated script, e.g.:

- Step animation: random block -> show H -> compute s -> give m -> compute d -> highlight -> verify;
- Quiz mode: generate random problems asking "which position flips?" with automatic scoring;
- Compare mode: embed the same message with LSB, matrix p3, and nsF5 p3, showing changes and Gn side by side.

10.4 Paper-to-Code Reading Table

Note: the paper draft has been updated to thesis/thesis1.pdf with v1.4.0 experiments (SRM, 143-D/53-D, multi-source data). The table below follows the earlier thesis numbering as an example.

Paper section	Corresponding code	Prereqs before reading

Paper section	Corresponding code	Prereqs before reading
Ch. 2 Background	ns5_core.py / steganalysis.py	LSB, Hamming, wet paper, chi-square/RS
Ch. 3 Requirements	README feature overview	Steganography vs steganalysis game view
Ch. 4 Design	architecture figure + ns5_core.py	Layers, hash-keyed pipeline
Ch. 4 Acceleration	cppembed.py / fsfeatures.py	ctypes and consistency self-checks
Ch. 4 GPU	gpu/*.py	Tensor ops, batching, throughput limits
Ch. 5 Experiments	run_e2e / make_dataset / train_model	GroupKFold, AUC, Youden

10.5 Common Defense / Presentation Questions

Frequent question	How to answer
Why is LSB hiding detectable?	Pair counts flatten (chi-square) and LSB structure collapses (RS); intuition first, statistics second
Why does matrix embedding change less?	n=2^p-1 positions carry p bits; distinct H columns locate every syndrome difference
What does nsF5 improve over F5?	Wet paper marks wet positions in advance and solves on dry ones - no shrinkage, no retries
How is the wet-paper equation solved?	GF(2) system H_dry*y=d; single column, then pairs, then Gaussian elimination
Why not a raw-pixel CNN?	Weak signals plus tiny independent-sample counts; features + simple models generalized better
What does AUC 0.75 vs 0.91 mean?	Ranking ability; AUC is threshold-independent while deployed detection needs an operating point
What does hash keying protect against?	Predictable/copyable positions and naive tampering; it is not a keyed MAC
Any risk from C++/GPU acceleration?	Yes - cross-language consistency checks are mandatory or embed/decode breaks

Frequent question	How to answer
Why keep 143d and 53d models?	143d is more robust OOD; 53d has higher AUC and full interpretability - different jobs

10.6 Delivery Checklist (Final Week)

- A code diff you can explain change by change;
- Experiment scripts plus raw outputs (no cherry-picking);
- Two or three figures (efficiency / ROC / density / tamper);
- A 3-5 page report: problem -> method -> results -> limits -> next steps;
- A 5-minute live demo from embedding to analysis;
- Ability to answer any three questions from 10.5 without notes.

Chapter 11 - Boundaries, Ethics, and What Comes Next (Read Along the Way)

11.1 Legal and Ethical Boundaries

Steganography is double-edged: it supports watermarking, media provenance, and covert authentication, but it can also be abused for malicious covert communication. The right posture for learning and research is:

- Experiment only on your own data, public datasets, or authorized material;
- Use the tooling for detection and forensics rather than concealment;
- Report limitations and false-positive risk honestly; never overstate detection power;
- Do not use other people's private photos as practice carriers; de-identify and authorize data.

> **Watch out |** The steganalysis half of this project (chi-square/RS/ML) is the shield: monitoring and forensics need it to find covert channels. Think about your data and purpose before studying attack surfaces.

11.2 Known Limitations (Understanding Boundaries Is Understanding the Project)

- Messages are ASCII-only by default; Chinese/UTF-8 requires an extension;
- Color images use only the R channel, underutilizing capacity;
- Keying is not a keyed MAC, so a strong attacker who re-embeds can update the header hash;

- The v1 dataset still starts from a limited set of independent photos (414 campus, later expanded with BOSSbase); weak-density and cross-source detection remain hard;
- The v2/143-D model was over-aggressive on out-of-distribution real JPEG clean images; the fix requires JPEG clean samples in training - always run OOD tests before deployment;
- SRM helps only within the same source domain and hurts across heterogeneous sources;
- The dual models trade off robustness and interpretability (143d: 1/8 OOD FP; 53d: higher AUC but 3/8 OOD FP); GUI switching is planned, use the API today;
- Heuristic analysis outputs a "stego tendency probability," not a strict statistical probability;
- Pixel-domain nsF5 is incompatible with lossy formats; JPEG-domain work needs the yccstego-style DCT coefficient design.

11.3 A Map of Modern Information Hiding and Steganalysis

Direction	Representative ideas / methods	Distance from an undergrad
Content-adaptive embedding	HUGO / WOW / UNIWARD: hide where texture is complex	Requires optimization background
Deep-learning steganography	Encoder-decoder models trained end-to-end	Reproducible entry projects exist
Deep-learning steganalysis	SRNet / large-scale CNNs	Needs big data and compute
JPEG-domain hiding	yccstego: nsF5 on quantized DCT coefficients of Y	Direct extension of this project

Direction	Representative ideas / methods	Distance from an undergrad
Robust/reversible watermarking	Trade invisibility, robustness, capacity	Engineering-heavy; industry-relevant

If your interest is sparked, the next steps are concrete: read the original wet-paper paper (Fridrich et al.), study yccstego's quantized-DCT pipeline, then pick one modern algorithm for a literature review - Appendix E gives entry points.

11.4 Closing Words

Twelve weeks ago steganography may have sounded like "hiding words in images." Now you can explain why it depends on carrier redundancy, why direct LSB changes leave statistical fingerprints, how Hamming codes carry the most information per modification, how wet paper coding removes shrinkage, and how machine learning states its confidence honestly. Those ideas connect one project in F:\Steganography to a way of thinking shared by information security, probability, and machine learning.

> **Try it |** Final exercise: without notes, write 500 words explaining to a classmate why nsF5 is better than naive LSB. Then open the README and compare. If you can write it, you have finished.

Appendix A - Glossary

Arranged roughly in reading order. Mastering the bold-faced terms covers almost everything in this handbook.

Term	Also known as	One-line meaning
Steganography	-	Hiding a secret inside an innocent carrier so the act of communication is hidden
Steganalysis	-	Deciding whether a carrier hides a secret, from statistics or learning
cover / stego	carrier / embedded image	Original image / image after embedding
LSB	least significant bit	Lowest bit of a pixel; flipping it is visually negligible
bit plane	-	Binary layer formed by one bit position across all pixels
redundancy	-	Carrier parts that can be modified without obvious artifacts
capacity / payload	embedding capacity	Maximum secret bits an image can carry
relative payload	-	Embedded bits divided by available positions
embedding efficiency	alpha	Average secret bits per modification, alpha
matrix embedding	-	Group coding that embeds p bits while changing at most one position
Hamming code	binary Hamming code	Linear error-locating code with block [n,k,3]
parity-check matrix	H matrix	p x n matrix with distinct columns used to compute syndromes
syndrome	-	s = H*x (mod 2); a summary of the block state
GF(2)	Galois field of two elements	Field containing only 0/1 with XOR arithmetic
shrinkage	-	Coefficient magnitude decrease lands on zero, ruining the block
wet paper coding	WPC	Embedding on dry positions while wet positions stay

Term	Also known as	One-line meaning
		untouched
dry / wet positions	-	Modifiable positions / untouchable positions
F5 / nsF5	-	JPEG matrix-embedding algorithm; nsF5 removes shrinkage with wet paper
magnitude decrease	coefficient shrink	Reducing |coefficient| by 1 to flip LSB parity
SHA-256	-	256-bit cryptographic hash used for image keying
keying	-	Using password/content keys to decide hiding positions
deterministic permutation	-	Shuffle where the same seed always gives the same order
splitmix64	-	64-bit PRNG used by the project
Fisher-Yates	-	Standard uniform shuffle algorithm
self-synchronization	-	Decoder finds the payload without external parameters
tamper perception	-	Detecting that an image was modified
blind steganalysis	-	Statistical detection without the original cover
chi-square test	Westfeld test	Checks whether gray-level pairs were balanced
p-value	-	Probability that observations fit the assumption; here high p is suspicious
RS analysis	-	Counts regular/singular groups under +/- masks
regular / singular	R / S groups	Groups whose discriminant rises / falls after flipping
difference entropy	-	Shannon entropy of adjacent differences; measures texture randomness
Shannon entropy	-	Information/uncertainty measure in bits
supervised learning	-	Training a model on labeled samples
feature vector	-	Numeric description of a sample (11-D v1 or 143-D v2

Term	Also known as	One-line meaning
		here)
label	-	Ground truth (0 = clean, 1 = stego)
binary classification	-	Predicting membership of one of two classes
logistic regression	-	Linear combination + sigmoid; an interpretable classifier
sigmoid	-	S-shaped function mapping any real number to 0-1
cross-entropy loss	-	Standard classification loss
gradient descent	-	Iteratively moving parameters downhill in loss
overfitting	-	Memorizing training data at the cost of generalization
cross-validation	-	Rotating validation folds to estimate generalization
data leakage	-	Validation information entering training, inflating metrics
GroupKFold	-	Grouped cross-validation (same photo's samples stay together)
confusion matrix	-	Four-cell counts of truth vs prediction
precision / recall	-	Share of flagged that are real / share of real that are caught
ROC / AUC	-	Detection-vs-FP curve across thresholds and its area
Youden's J	-	Threshold maximizing TPR - FPR
false positive / negative	FP / FN	Clean flagged as stego / stego missed
SRM	Spatial Rich Model	High-pass residual filter family that highlights embedding noise
residual map	-	Noise residual after high-pass filtering
LightGBM / LGB	-	Gradient-boosting implementation used by the v1.4 models
feature group	-	A batch of features from one source

Term	Also known as	One-line meaning
		(BASE/SRM/PREFIX/LSB-PREFIX)
out-of-distribution	OOD	Inputs unlike the training distribution (e.g., real JPEG clean)
BOSSbase	BOSSbase 1.01	Standard benchmark: 10,000 512x512 grayscale PGM images
PGM	-	Lossless grayscale image format (BOSSbase carrier format)
stacking	meta-learner	Ensemble where a second model combines base-model predictions
memmap	memory-mapped file	Large arrays written/read in slices to avoid OOM
calibration set	-	Separate subset used only for threshold selection
grid tuning	-	Searching a hyperparameter grid and comparing models
dual-version model	-	143d robust model + 53d interpretable model deployed together
ctypes	-	Python binding mechanism for C/C++ DLLs
DLL	dynamic-link library	Windows shared library
CUDA / batching	-	GPU parallelism / processing many samples at once
consistency check	-	Bit-level comparison between C++/GPU and Python outputs

Appendix B - Command Cheat Sheet

Run from the project root F:\Steganography.

Purpose	Command
Install core dependencies	pip install numpy pillow
Install everything (ML/plot)	pip install -e .
Core algorithm self-test	python src\test_core.py
Steganalysis self-test	python src\test_steg.py
False-positive regression	python src\test_false_positive.py
GUI smoke test	python src\test_gui.py
End-to-end verification	python src\run_e2e.py
Launch the GUI	python src\gui.py
Code-family/efficiency plot	python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"
Generate v1 dataset	python src\make_dataset.py data\campus_jpg --out campus
Generate v2 dataset (12 variants / 143-D)	python src\make_dataset.py --out campus_v2 --feature-set v2 --variants all
Generate SRM-enhanced dataset (same source)	python src\make_dataset.py data\campus_jpg --out campus_srm --preprocess srm -j 16
Train/evaluate a dataset	$env:DS_FILES = "dataset.csv"; python src\train_model.py
Train on v2 + JPEG clean	$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py
Single-image ML prediction	python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img/cover.png').convert('L'))))"
Switch to 53d interpretable model	python -c "import sys; sys.path.insert(0,'src'); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path='models/stego_classifier_v2_jpeg_lgb_51d.jobl

Purpose	Command
	ib', clip_outliers=False).predict(np.asarray(Image.open('img/cover.png').convert('L'))))"
C++/Python consistency check	python -c "import sys; sys.path.insert(0,'src'); import cppembed; cppembed.selfcheck()"
GPU dataset generation	python gpu\make_imageset.py
GPU feature self-check	python gpu\featurize_gpu.py
GPU v2 feature self-check	python gpu\featurize_v2_gpu.py
GPU training	python gpu\train_ml_gpu.py
GPU single-image detection	python gpu\predict_gpu.py img\cover.png
BOSSbase multi-source (GPU)	python gpu\make_imageset.py data\BOSSbase_1.01 --out bossbase, then python gpu\train_ml_gpu.py
Build packages	python -m build
Install wheel	pip install dist\nsf5stego-1.4.0-py3-none-any.whl

Appendix C - 12-Week Checklist

Week	Topic	Required action	Date
1	Images & binary	Run the bit operations; state what LSB means	
2	Python environment	pip install; run run_e2e.py; save one grayscale image	
3	LSB & blind analysis	Embed -> analyze experiment; read test_steg.py output	
4	Chi-square / RS	Explain Gn collapse to a friend; run test_false_positive.py	
5	Matrix embedding / F5	Work one p=3 example; verify with matrix_demo	
6	nsF5 / wet paper	Read the wet-paper section of ns5_core.py; run its test	
7	Hash keying	Wrong-password and one-pixel experiments; write the keying boundary	
8	ML foundations	Run v1 training; explain GroupKFold and AUC	
9	ML steganalysis	Compare 11-D / 143-D / 53-D predictions; explain SRM and dual models	
10	Engineering	Run cppembed.selfcheck and featurize_v2_gpu; read SRM/multi-source pipeline	
11	Capstone	Choose a direction and finish 70% of code/experiments	
12	Presentation	Finish report and demo; answer three questions from 10.5 unaided	

Appendix D - Concept-to-Code Map

Concept	File	Read first
Message encoding / length header	src/ns5_core.py	encode_string / decode_string
Image hash / seeds	src/ns5_core.py	derive_seed / get_image_hash
Deterministic permutation	src/ns5_core.py	permute_index
Hamming code / syndrome	src/ns5_core.py	build_hamming / syndrome
Matrix embedding	src/ns5_core.py	MatrixEmbedding._embed
Wet paper solver	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
Pixel-domain nsF5	src/ns5_core.py	nsF5Pixel._embed
High-level embed/extract	src/ns5_core.py	embed_string / extract_string
Chi-square statistics	src/steganalysis.py	chi2_stats / chi2_sf
RS statistics	src/steganalysis.py	rs_metrics
Combined verdict	src/steganalysis.py	analyze
v1 feature order	src/fsfeatures.py / ml_predict.py	FEAT_KEYS / V1_FEAT_KEYS
v2 143-D features	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM preprocessing	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch
Training pipeline	src/train_model.py	main()
Dataset generation	src/make_dataset.py	main()
C++ embedding wrapper	src/cppembed.py	embed_string / selfcheck
Dual-model inference	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)
GPU v1 features	gpu/featurize_gpu.py	batch operators / check_vs_cpu
GPU v2 features	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() / self-check

Concept	File	Read first
GUI callbacks	src/gui.py	button events -> core functions

Appendix E - Recommended Resources

Books

- Zhou Zhihua, Machine Learning (in Chinese) - especially Chapter 2 on model evaluation and selection;
- Hang Li, Statistical Learning Methods (in Chinese) - classic derivations of logistic regression and perceptrons;
- Gonzalez & Woods, Digital Image Processing - bit planes, histograms, and frequency-domain fundamentals (early experiments used its images; v1.4 data sources switched to campus photos and BOSSbase);
- Any Chinese textbook on information hiding and digital watermarking as broad reading.

Core Papers (in reading order)

- Westfeld, F5 - A Steganographic Algorithm, 2001 - matrix embedding and F5;
- Westfeld & Pfitzmann, Attacks on Steganographic Systems, 1999 - chi-square test;
- Fridrich, Goljan & Du, Reliable Detection of LSB Steganography in Color and Grayscale Images, 2001 - RS analysis;
- Fridrich, Goljan, Lisonek & Soukal, Writing on Wet Paper, 2005 - wet paper coding;
- Pevny, Filler & Bas, Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis - modern feature-based steganalysis (SPAM) entry point.

Online

- The project README and the thesis draft thesis/thesis1.pdf (SRM / 143-D / 53-D experiments are in the experimental chapter); GitHub: Yukinoshita-lin/nsf5-steganography;

- scikit-learn docs: LogisticRegression, GroupKFold, roc_curve;
- PyTorch tutorials (needed for the GPU pipeline);
- GitHub Actions documentation for CI/release.

> **Try it |** The handbook ends here. Return to the checklist in 0.4 and grade yourself item by item. If you can tick all of them, give yourself a certificate: nsF5 Steganography - From Zero to Understanding.